Implementation Details

Event Structure (event.h)

- 64 bytes, `#pragma pack(1)` for tight packing
- `uint64_t nano_stamp` : nanosecond timestamp
- `uint64_t sequence_number` : event identifier
- `uint64_t payload_size` : payload byte count
- `uint8_t payload(40)` : embedded payload buffer
- `static_assert(sizeof(Event) == 64)`

RingBuffer (ring_buffer.h)

- Template: `RingBuffer<uint32_t Capacity>` — Capacity must be power of two
- `Event buffer(Capacity)` : fixed-size circular buffer
- `std::atomic<uint32_t> write_index` : producer position
- `std::atomic<uint32_t> read_index` : consumer position
- `static constexpr uint32_t MASK = Capacity - 1` : bitwise wrapping
- Methods:
    - `bool push(const Event&)` : write event, release semantics, returns false if full
    - `bool pop(Event&)` : read event, acquire semantics, returns false if empty
    - `bool is_empty() const` : check if buffer has no events
    - `bool is_full() const` : check if buffer cannot accept events
    - `uint32_t size() const` : return count of events in buffer
- Index wrapping uses bitwise AND `& MASK` (one CPU cycle)
- Full check: `(next_write & MASK) == read_idx`
- Empty check: `write_idx == read_idx`
- `static_assert` enforces power-of-two capacity at compile time

MemoryPool (memory_pool.h)

- Struct: `Handle` with index and generation fields
- Template: `MemoryPool<typename T, uint32_t Capacity>`
- `std::array<T, Capacity> storage` : pre-allocated objects
- `std::array<uint32_t, Capacity> generations` : version counter per slot
- `std::array<bool, Capacity> available` : free slot tracking (default filled with true)
- Methods:
    - `Handle allocate()` : find first available slot, increment generation, mark unavailable, return Handle
    - `void deallocate(Handle)` : validate generation matches, mark slot available
    - `T* get(Handle)` : validate generation matches, return pointer to object or nullptr
- Generation validation catches use-after-free at runtime

EventBus (event_bus.h)

- Template: `EventBus<uint32_t BufferCapacity, uint32_t PayloadCapacity>`
- `RingBuffer<BufferCapacity> command_buffer` : inbound events
- `RingBuffer<BufferCapacity> response_buffer` : outbound events
- `MemoryPool<uint8_t, PayloadCapacity> payload_pool` : payload storage (unused in Phase 1)
- `uint32_t latency_ns` : fixed delay simulation
- `uint32_t jitter_ns` : random delay variation
- `float packet_loss_rate` : drop probability (0.0 to 1.0)
- Methods:
    - `void send(const Event&)` : apply latency/jitter/loss, write to command buffer
    - `bool receive(Event&)` : apply latency/jitter/loss, read from response buffer
    - `Handle allocate_payload()` : allocate slot in payload pool
    - `uint8_t* get_payload(Handle)` : retrieve payload pointer
- Payload embedded directly in Event structure (40 bytes)
- Latency and jitter simulation uses `<random>` and `std::this_thread::sleep_for()`

Phase 1 Performance Results

- **Latency:** 3.7µs median, 26ns minimum, 95µs P99 (1M events, 65KB buffer)
- **Throughput:** 13.6M events/sec (target: 1M/sec, exceeds by 13x)
- **Distribution:** 33% of events complete sub-1µs, 56% sub-10µs, 99% sub-100µs
- **Benchmark:** 1 million events, power-of-two 65536-byte buffer, concurrent producer/consumer
- Note: 500ns target requires CPU affinity and dedicated cores (deferred to Phase 2)

```
Number of Events:   1000000.
Buffer Size:        1024*64 bytes.

Average latency:    19960.2 ns
Min latency:        26 ns
Max latency:        243530 ns

P50 (median):       3767 ns
P90:                59373 ns
P95:                69157 ns
P99:                94893 ns
P99.9:              220587 ns

Throughput:         1.35643e+07 events/sec
Total time:         0.0737232 seconds
```

Phase 1 Complete

- Lock-free SPSC ring buffer with acquire/release semantics
- Power-of-two capacity with bitwise AND masking
- Zero dynamic allocation in critical path
- Comprehensive performance benchmark with percentile analysis
- Ready for Phase 2 integration